

UNIVERSITY OF HAIFA

Edge-Centric Intrusion Detection: A Critical Data Science Evaluation

Ilay Nadler

Supervisor: Dr. Uri Itai

July 2026

Abstract

This report critically evaluates the methodology and claims presented in "Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network" (Singh et al., 2021). The original authors proposed a lightweight, sliding-window Recurrent Neural Network (RNN) to detect Distributed Denial-of-Service (DDoS) attacks on resource-constrained edge devices, claiming up to 99% accuracy on the UNSW-NB15 dataset. During the reproduction process, a major discrepancy was discovered: the dataset officially released for reproduction was not UNSW-NB15, but rather a specific, highly imbalanced capture from the CICIDS2017 benchmark. Furthermore, comparative benchmarking revealed that a stateless Random Forest baseline outperformed the proposed Edge-Detect architecture in both predictive metrics and inference speed. Ultimately, this report demonstrates that while the proposed RNN minimizes disk footprint, its temporal complexity is computationally wasteful for this feature space, and its high accuracy is largely an artifact of a 56.7% synthetic sampling bias that would cause severe alert fatigue in real-world edge deployments.

Contents

1	Executive Summary	2
2	Summary of the Source	2
3	Critical Evaluation	3
4	Exploratory Data Analysis	3
5	Feature Engineering Analysis	4
6	Reproducibility Analysis	4
7	Experimental Results	5
7.1	Evaluation Metrics	5
7.1.1	Confusion Matrix	5
7.1.2	Matthews Correlation Coefficient (MCC)	5
7.2	Obtained Results	5
8	Conclusions and Error Analysis	6
8.1	Error Analysis	6
8.2	Conclusions	6
9	Summing It Up	6

1 Executive Summary

As per the project guidelines, this section provides a concise summary of the entire project. This report critically evaluates the paper "Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network," which proposes a lightweight recurrent (LSTM/GRU-based) architecture for detecting Distributed Denial-of-Service (DDoS) attacks on resource-constrained edge devices. The project investigates whether the authors' claims of achieving up to 99% accuracy with a minimal model footprint are genuinely supported by the data.

The paper states that the authors evaluated their model on the UNSW-NB15 dataset. However, reproducing the work from the authors' own public repository revealed that the artifact actually shipped for reproduction is not UNSW-NB15 at all: it is the equally well-known CICIDS2017 Friday-WorkingHours-Afternoon-DDoS.pcap_ISCX.csv file. This discrepancy between the paper's stated methodology and its released reproducible artifact is undocumented and is, in itself, a central reproducibility finding of this report. The main issue about this is not the gap in reproducibility, but the fact that the modified dataset being used in the implementation has unrealistic prevalence, where the majority of samples are DDoS attacks.

Key Findings: The Random Forest baseline outperformed the proposed Edge-Detect RNN architecture in both predictive metrics and inference latency, demonstrating that the complex sliding-window methodology is computationally wasteful for this feature space. Furthermore, the released dataset carries a 56.7% DDoS prevalence, since it is an isolated capture of a single attack afternoon rather than a representative slice of edge traffic; a model calibrated on this file alone would generate excessive False Positives if deployed against realistically benign-heavy edge traffic.

2 Summary of the Source

The paper "Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network" by Singh et al. tackles Network Intrusion Detection in the specific domain of edge-centric networks.

An *intrusion detection system* (IDS) is an application that sits on top of a network or some system to monitor malicious activity [5]. It may be a central computer that collects data from all nodes of a network to be aware of all of it at once, or it may sit at critical sections and monitor locally. It differs from a *firewall* by the way it detects anomalies: The original firewall operates by preset, static rules for preventing suspicious activity from entering the network, while the IDS monitors the activity inside the network and alerts potential maliciousness. By its nature, it is more dynamic.

In particularly large networks, IDS is highly influential, especially in comparison to more traditional security measures like firewalls, because:

- Networks are distributed and data cannot move from all nodes to a central computer in a reasonable time.
- Activities that are interpreted as threats in one part of the network can be normal in another.
- The network being a dynamic structure requires highly dynamic measures for security.

The paper tackles the challenge of fast and dynamic, distributed IDS. It proposes a framework to detect DDoS attacks that is composed of many lightweight Deep Learning (DL) models that sit on the edges between nodes of the network. It consists of training sequential learning models such as RNN, LSTM, and GRU to do so in little time and with few computational resources.

Stated dataset (per the paper): The authors state that they selected the UNSW-NB15 dataset for their evaluation [1], citing its relative modernity, clean training/testing separation, and correctly labelled features. UNSW-NB15 has 49 native features [3].

Actual dataset (per the released artifact): As detailed in the Reproducibility Analysis, the authors’ repository does not contain UNSW-NB15. It contains `Friday-WorkingHours-Afternoon-DDos.pcap_` a well-known file from the CICIDS2017 benchmark [4]. This file has 78 CICFlowMeter-derived features plus a binary label, and 225,745 rows. This reproduction study necessarily works from what the authors actually released.

3 Critical Evaluation

The core claim made by the authors is that their proposed recurrent pipeline can achieve highly reliable classification accuracy (up to 99%) while retaining a model size small enough to execute on resource-constrained devices like a Raspberry Pi.

The dataset claim does not match the released artifact: Before evaluating the accuracy claim itself, a fundamental issue emerged. A reader attempting to verify the paper’s central claim using the authors’ own released code and data would be testing it against a dataset the paper never claims to use (CICIDS2017 instead of UNSW-NB15), with no note in the documentation explaining the substitution.

To critically evaluate the accuracy claim itself, we must investigate whether the reported number reflects genuine predictive power or an artifact of class composition. In edge networks, benign traffic naturally outweighs malicious traffic. The CICIDS2017 file actually released is an isolated capture of a single Friday afternoon during which a DDoS exercise was specifically staged. Its 56.7% DDoS composition reflects the choice of capture window, not a property of network traffic in general.

Evaluation of Author’s Methodology: The evaluation methodology available for reproduction is fundamentally mismatched to a real edge deployment. In a real-world edge environment where benign traffic accounts for $>99\%$ of all flows, a model calibrated on an inverted dataset will generate substantial False Positives. Furthermore, the authors’ architectural claim that a complex sliding-window RNN ($T = 10$) is necessary to capture temporal attack states is contradicted by evidence. Our evaluation proves that a stateless Random Forest baseline achieved superior predictive performance, indicating that the computationally expensive sequential transformation is unnecessary.

4 Exploratory Data Analysis

Before any feature engineering decisions were made, the loaded partition was examined.

Distributions and outliers: The numerical flow-based features (e.g., `Flow Duration`, `Flow Packets/s`) are heavily right-skewed, with a long tail of extreme values consistent with burst

traffic during an attack. Boxplot inspection confirmed a meaningful number of statistical outliers; however, in a network intrusion context, these outliers are frequently the very signal that separates an attack flow from benign traffic, so they were retained.

Missing values: A leading `ID_Column` mismatch between the shipped header file and the data file first had to be corrected. Once aligned, only the `Flow Bytes/s` column contained conventional missing values. Several flow-rate columns additionally contained `Infinity` values, produced by division-by-zero when flow duration is 0. Both were treated as missing and dropped, reducing the dataset from 225,745 to 225,711 rows.

Class imbalance (prevalence): The dataset carries a 56.72% DDoS / 43.28% Benign split. In real-world terms, an edge gateway trained on this distribution will learn a decision boundary calibrated for a world where roughly half of all traffic is malicious.

Correlation analysis: Spearman correlation was chosen over Pearson because several flow-based features are monotonically but non-linearly related. Pairs with $|\rho| > 0.9$ were flagged as redundant, identifying 38 of the 78 features.

5 Feature Engineering Analysis

Unlike UNSW-NB15, this CICIDS2017 file contains no string-valued categorical columns: all 78 `CICFlowMeter` features are already numeric. Standard scaling (Z-score normalization) was applied to ensure features with massive value ranges did not mathematically overpower smaller metrics during gradient descent.

The Spearman correlation analysis in the Exploratory Data Analysis section identified 38 of the 78 features as highly redundant ($|\rho| > 0.9$). These 38 redundant features were successfully dropped prior to training. This mathematical reduction of the dimensionality significantly decreased the computational footprint of both models while maintaining near-perfect predictive accuracy, which is highly desirable for edge computing environments.

A crucial hidden preprocessing step involved addressing division-by-zero errors in flow rate features (e.g., `Flow Packets/s`). These generated `Infinity` values which crash backpropagation; these rows were programmatically isolated and dropped.

6 Reproducibility Analysis

The official implementation for the "Edge-Detect" paper is hosted on GitHub under the repository <https://github.com/racsa-lab/Edge-Detect> [2].

Dataset mismatch: The paper's text states that the authors chose the UNSW2015 dataset. The repository's `dataset/` folder, however, contains a raw CICIDS2017 file. This file has exactly 225,745 rows and 78 features—an exact match to the publicly documented composition of the CICIDS2017 Friday-Afternoon-DDoS file.

Execution and Dependencies: The fundamental architecture was successfully reconstructed and executed within a Python/TensorFlow environment via Google Colab. The sliding window transformations mapped exactly to the theoretical dimensions described in the paper.

Hidden Preprocessing: Loading the released files revealed a header-alignment bug: `ddos_column_headers` lists one fewer column name than `modified_dataset.csv` actually has, requiring a manually in-

serted leading `ID.Column` name to align the two. Beyond this, no further hidden transformations were found.

7 Experimental Results

Two models were trained: the authors’ proposed deep learning approach (RNN) and a traditional machine learning baseline (Random Forest).

7.1 Evaluation Metrics

Evaluating edge-centric intrusion detection requires moving beyond simple accuracy due to class imbalance. Accuracy, Precision, and Recall were computed but both models scored ≈ 1.00 across these metrics due to task saturation. MCC and the confusion matrix are the metrics that actually distinguish the two models.

7.1.1 Confusion Matrix

The confusion matrix breaks predictions into True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

	Edge-Detect (GRU, T=10)		Random Forest	
	Pred. Attack	Pred. Normal	Pred. Attack	Pred. Normal
Actual Attack	TP = 25581	FN = 19	TP = 25602	FN = 3
Actual Normal	FP = 8	TN = 19526	FP = 2	TN = 19536

7.1.2 Matthews Correlation Coefficient (MCC)

The MCC measures the correlation between the observed and predicted binary classifications. Because MCC incorporates all four values of the confusion matrix, it only yields a high score if the model performs well on both benign traffic and attacks.

7.2 Obtained Results

The baseline Random Forest classifier outperformed the proposed Edge-Detect reproduction on MCC (0.9998 vs. 0.9988) and on the confusion matrix breakdown (2 FP and 3 FN, versus 8 FP and 19 FN for the GRU-based model).

To definitively test the authors’ core claim regarding edge constraints, a computational resource benchmark was executed. The proposed model achieved a highly compressed disk footprint of 0.16 MB compared to the Random Forest’s 1.33 MB. However, the Random Forest demonstrated vastly superior inference speed, requiring only 0.0124 ms per packet compared to 0.4613 ms per temporal window for the sliding-window model.

8 Conclusions and Error Analysis

8.1 Error Analysis

In intrusion detection, a False Negative allows a payload to successfully bypass the IDS, potentially causing complete network downtime. Conversely, excessive False Positives can overwhelm security administrators with alert fatigue.

Because the only dataset released for reproduction carries a 56.7% DDoS prevalence, a model's internal decision threshold calibrated on it is aggressively biased toward predicting an attack. If deployed in an authentic edge network (where $> 99\%$ of flows are benign), this bias would guarantee a cascade of False Positives far beyond what the near-perfect test scores suggest.

8.2 Conclusions

The most significant finding of this reproduction is that the paper's stated dataset (UNSW-NB15) and its released reproducible artifact (CICIDS2017) do not match.

Separately, the original authors justified their complex, sliding-window RNN architecture by claiming it utilizes low computational resources suitable for edge nodes. While the Edge-Detect reproduction successfully minimizes disk footprint (0.16 MB), its architecture introduces unjustified computational latency. A stateless Random Forest provides faster, more accurate predictions that easily fit within the standard memory constraints (1-5 MB) of modern edge gateways like a Raspberry Pi.

9 Summing It Up

This project evaluated the viability of edge-centric deep neural networks for network intrusion detection. Attempting to reproduce the paper "Edge-Detect" surfaced a major dataset discrepancy.

Based on the critical evaluation, the authors' original claims were not fully supported by the evidence. While the model was highly compressed (0.16 MB), the traditional Random Forest baseline provided a better MCC (0.9998 vs. 0.9988) and significantly faster inference latency (0.0124 ms/packet vs 0.4613 ms/window), proving that the complex sliding-window sequential transformation was computationally wasteful. Ultimately, deploying this specific implementation in a real-world scenario is **NOT RECOMMENDED** without first resolving the dataset discrepancy, addressing the resulting sampling bias, and removing the unnecessary temporal complexity of the model architecture.

References

- [1] P. Singh, J. J. P. A. Pankaj, and R. Mitra, "Edge-Detect: Edge-Centric Network Intrusion Detection using Deep Neural Network," in *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*, pp. 1-6, 2021. <https://doi.org/10.1109/CCNC49032.2021.9369469>

- [2] racsa-lab, “Edge-Detect” [GitHub repository], <https://github.com/racsa-lab/Edge-Detect>
- [3] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” in *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1-6, 2015.
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108-116, 2018.
- [5] Wikipedia contributors, “Intrusion detection system,” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Intrusion_detection_system